JARVIS OS ��� README

A modular, offline-first voice assistant that runs your Windows PC — hands-free.
Wake word → speech-to-text → intent → action → neural voice reply, all on your own machine.

What is JARVIS?

JARVIS is a desktop AI assistant for Windows 11 that you talk to. Say the wake word **"Hey Jarvis"**, give a command in plain English, and it opens apps, controls volume and brightness, browses the web, plays music, manages files, launches your coding workspace or games, remembers context, and answers free-form questions through a local LLM — then replies in a natural neural voice.

Everything runs **locally**: speech recognition, text-to-speech, and the language model all execute on your machine, so nothing leaves your PC.

Highlights

- **Wake-word activation** — a gapless rolling-window listener spots "Hey Jarvis" continuously, so you don't have to time your speech to a recording gap.
- **Offline speech-to-text** — `faster-whisper` with automatic GPU→CPU fallback, so a missing CUDA runtime never stops it from working.
- **Neural text-to-speech** — Piper voice for human-sounding replies, with SAPI5 and pyttsx3 as automatic fallbacks.
- **Local AI brain** — Ollama (`qwen2.5` family) handles anything the fixed commands don't, so you can speak naturally instead of memorising phrases.
- **Risk-gated actions** — every command is classified SAFE / MEDIUM / HIGH; destructive actions ask for confirmation, and power actions ship in SAFE MODE.
- **Persistent memory** — SQLite-backed context so references like "open that project again" resolve across sessions.
- **GUI dashboard** — a Tkinter window with live status, chat log, and recent actions, plus an optional system-tray mode.
- **Fully modular** — 14 subsystems behind clean base classes; swap the STT, TTS, or LLM engine without touching the rest.

Architecture at a glance

```
Microphone ─► Wake Word ─► STT (Whisper) ─► Router ─► Command / AI Brain ─► Action
                                              │                                 │
                                         Risk Gate ◄── Confirmation      Memory (SQLite)
                                              │
                                         TTS (Piper) ─► Speaker
```

Layer	Module	Role
Voice I/O	`voice/`	Mic capture, wake word, STT, TTS, voice channel
Core	`core/`	Assistant lifecycle, session loop, router, risk model
Brain	`brain/`	Command registry, intent parsing, LLM fallback
Automation	`automation/`	Windows, browser, dev, games, files control
Memory	`memory/`	SQLite store, context resolution
Security	`security/`	Confirmation gating for risky actions
GUI	`gui/`	Tkinter dashboard + tray
Config / Utils	`config/` , `utils/`	Settings, logging, text helpers

Capabilities

Domain	Examples you can say
Apps	"open notepad", "close chrome"

System	"volume 30", "brightness 50", "mute"
Web	"open youtube", "play arijit singh", "search python decorators"
Coding	"open my project in vs code", "run the project", "create a venv"
Gaming	"open steam", "launch "
Files	"find my resume", "open downloads", "organize this folder"
Memory	"remember my roll number is ...", "what is my roll number"
Conversation	anything else → answered by the local LLM

Requirements

- **Windows 11**, **Python 3.14**
- A working microphone and speakers
- ~2 GB free disk (Whisper + Piper + one Ollama model)
- **Ollama** for the AI brain — https://ollama.com/download
- *(optional)* NVIDIA GPU + CUDA runtime for faster STT

Installation

```
# 1. clone
git clone https://github.com/manthandhanraj/Jarvis-OS.git
cd Jarvis-OS

# 2. virtual environment
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 3. dependencies
pip install -r requirements.txt

# 4. neural voice (Piper)
mkdir models\piper
python -m piper.download_voices en_GB-alan-medium --download-dir models\piper

# 5. AI brain (Ollama) — install from ollama.com, then:
ollama pull qwen2.5:3b-instruct
```

Usage

```
python main.py                # text mode (type commands)
python main.py --mode voice   # voice mode (say "Hey Jarvis")
python main.py --mode gui     # dashboard window

# flags
python main.py --no-ai        # disable the LLM (keyword commands only)
python main.py --no-memory    # disable persistent memory
```

Say **"Hey Jarvis"**, wait for the acknowledgement, then speak your command.

Safety model

Actions are classified by risk and gated accordingly:

- **SAFE** — apps, volume, brightness, web, search: run immediately.
- **MEDIUM / HIGH** — destructive or system-level: require spoken confirmation.
- **Power actions** (shutdown / restart / sign-out) ship in **SAFE MODE** — they are simulated unless explicitly armed with the `JARVIS_ARM_POWER=1` environment variable, after an incident where a test command powered off the machine.

Configuration

All tunables live in `config/settings.py` — STT model size and language, mic sensitivity and endpointing, wake-word window, TTS

rate/voice, LLM model and endpoint, memory database path, and per-domain automation settings.

Roadmap

- [ ] Day 6 — voice authentication (speaker verification)
- [ ] GPU-accelerated STT preset (`small` / `medium` on CUDA)
- [ ] Richer conversational memory
- [ ] Packaged installer

Tech stack

`Python 3.14` · `faster-whisper` · `Piper` · `Ollama` · `sounddevice` · `comtypes / SAPI5` · `SQLite` · `Tkinter`

License

Personal project by **Manthan Dhanraj**. All rights reserved unless a license file is added.

"*Sometimes you gotta run before you can walk.*" — *Tony Stark*